

LAB TASKS

Task 1

Convert the following searching algorithms into agent-based models:

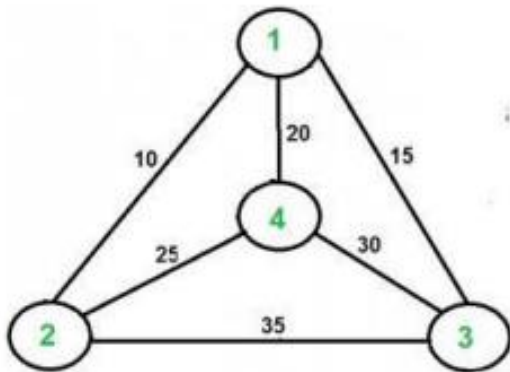
- **Depth-Limited Search (DLS):** Implement as a Goal-Based Agent to explore the graph up to a specified depth limit.
- **Uniform Cost Search (UCS):** Implement as a Utility-Based Agent to find the goal with the minimum cost path.

TASK # 2

Traveling Salesman Problem:

Given a set of cities and distances between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point. Like any problem, which can be optimized, there must be a cost function. In the context of TSP, total distance traveled must be reduced as much as possible.

Consider the below matrix representing the distances (Cost) between the cities. Find the shortest possible route that visits every city exactly once and returns to the starting point.



Task 3

Description: Implement UCS (Uniform Cost Search) to find the cheapest path in a graph where nodes represent locations, and edges represent roads with costs (e.g., fuel, time, distance). The algorithm should manage a priority queue to explore the least expensive path.

Graph structure: The graph could be sparse or dense, and UCS must handle different graph structures efficiently.

Cycle management: The algorithm must handle cycles (where nodes are revisited) and avoid exploring them unnecessarily.

Cost optimization: UCS must correctly manage a priority queue to expand the least costly path and ensure the optimal path is found.

Task 4

Description: Implement DLS to solve the 8-Puzzle (sliding tile puzzle) with a depth limit. The puzzle has 8 numbered tiles and one empty space. The agent must find the optimal solution path to arrange the tiles in ascending order within the given depth limit.

Depth limit: The algorithm must search for the solution within a fixed depth limit. If no solution is found within that limit, the algorithm should report failure.

Large state space: The state space for the 8-puzzle grows quickly, making it difficult to explore completely.

Optimality: Ensure that the solution path is optimal, meaning the fewest possible moves, even though you have a fixed depth limit.

Task 5

Description: Implement IDFS for a decision-making problem where an AI agent needs to make decisions based on changing conditions and evolving constraints over time. The agent needs to decide which tasks to complete in each stage, considering resource limitations, time constraints, and goal dependencies.

Dynamic constraints: At each depth level, new constraints are applied (e.g., limited resources, changing goals), requiring the agent to reconsider prior decisions.

Large state space: The decision tree grows exponentially, and IDFS must manage both time and memory constraints to explore deeper levels progressively.

Goal dependencies: Each decision the agent makes may affect future goals or constraints, making it necessary to adjust the agent's strategy dynamically.

Task 6

Description: Implement DFS to solve a Sudoku puzzle, but with an added complexity of constraint propagation. The algorithm should efficiently prune the search space and backtrack when an invalid configuration is encountered, minimizing unnecessary steps. Each time a number is placed in the puzzle, propagate the constraints to eliminate possible values in the affected rows, columns, and 3x3 grids.

Constraint propagation: As you place a number, propagate the constraint by eliminating that number from the possible values in the affected rows, columns, and 3x3 grids.

Efficient Backtracking: Minimize the number of recursive calls by checking the puzzle constraints early and backtracking only when necessary.

Complexity: The algorithm should be optimized in terms of time complexity by pruning search branches where possible.